CS2023 Algoritmos y Estructuras de Datos
Pilas, Colas y Colas dobles
Semana 4

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

31 de agosto de 2026

¿Qué aprenderemos hoy?

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Pila (*Stack*): LIFO

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Pila (*Stack*): LIFO
- El TDA Cola (*Queue*): FIFO

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Pila (*Stack*): LIFO
- El TDA Cola (*Queue*): FIFO
- El TDA Cola Doble (*Deque*)

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Pila (*Stack*): LIFO
- El TDA Cola (*Queue*): FIFO
- El TDA Cola Doble (*Deque*)
- Comparando los tres

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El TDA Pila (*Stack*): LIFO
- El TDA Cola (*Queue*): FIFO
- El TDA Cola Doble (*Deque*)
- Comparando los tres
- Aplicaciones clásicas

Repaso: TDA

Repaso rápido: ¿qué es un TDA?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De la semana pasada

Un **Tipo de Dato Abstracto** fija **qué** operaciones existen y qué hacen (la interfaz), sin decir **cómo** están implementadas por dentro.

Repaso rápido: ¿qué es un TDA?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De la semana pasada

Un **Tipo de Dato Abstracto** fija **qué** operaciones existen y qué hacen (la interfaz), sin decir **cómo** están implementadas por dentro.

Hoy: dos TDAs nuevos, más una variante

Ya vimos el TDA Lista (arreglo, vector). Hoy vemos dos TDAs con una interfaz mucho más **restringida**: no dejan acceder a cualquier posición, solo a los "extremos". Esa restricción, lejos de ser una limitación, es lo que los hace útiles.

TDA Pila (Stack)

El TDA Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una **pila** (*stack*) es una colección donde solo se puede insertar y eliminar por **un mismo extremo**, llamado el **tope** (*top*).

El TDA Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una **pila** (*stack*) es una colección donde solo se puede insertar y eliminar por **un mismo extremo**, llamado el **tope** (*top*).

LIFO: *Last In, First Out*

El último elemento en entrar es siempre el primero en salir — como una pila física de platos: solo puedes agregar o quitar el de **arriba**.

Operaciones del TDA Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Efecto	Complejidad exigida

Operaciones del TDA Pila

Operación	Efecto	Complejidad exigida
`push(x)`	agrega x en el tope	$O(1)$

Operaciones del TDA Pila

Operación	Efecto	Complejidad exigida
`push(x)`	agrega x en el tope	$O(1)$
`pop()`	quita el elemento del tope	$O(1)$

Operaciones del TDA Pila

Operación	Efecto	Complejidad exigida
`push(x)`	agrega x en el tope	$O(1)$
`pop()`	quita el elemento del tope	$O(1)$
`top()`	consulta (sin quitar) el tope	$O(1)$

Operaciones del TDA Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	**Efecto**	**Complejidad exigida**
`push(x)`	agrega x en el tope	$O(1)$
`pop()`	quita el elemento del tope	$O(1)$
`top()`	consulta (sin quitar) el tope	$O(1)$
`empty()`	¿está vacía?	$O(1)$

Operaciones del TDA Pila

Operación	Efecto	Complejidad exigida
`push(x)`	agrega x en el tope	$O(1)$
`pop()`	quita el elemento del tope	$O(1)$
`top()`	consulta (sin quitar) el tope	$O(1)$
`empty()`	¿está vacía?	$O(1)$

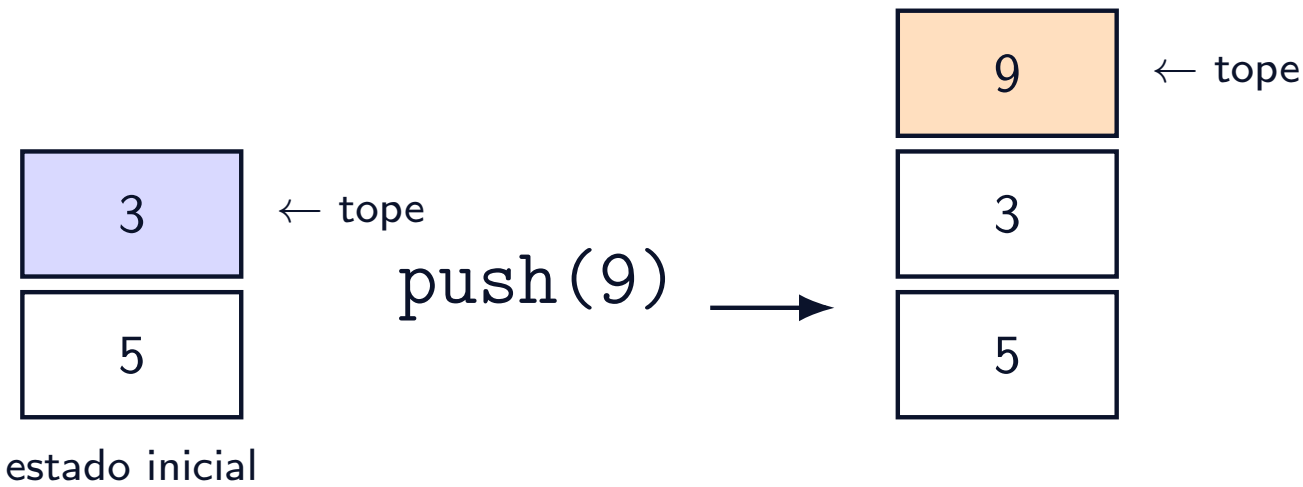
Idea clave

El TDA **exige** que las cuatro operaciones sean $O(1)$ — eso es parte del **contrato**, sin importar todavía cómo se implemente por dentro (lo veremos la próxima clase).

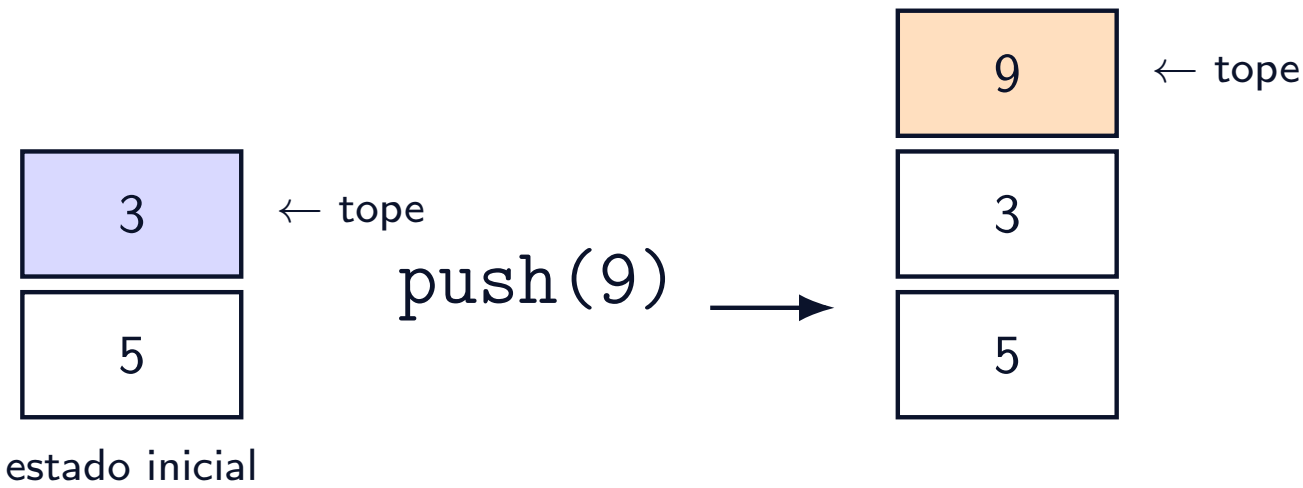
Visualizando `push` y `pop`



Visualizando `push` y `pop`



Visualizando `push` y `pop`



`push` solo toca el tope: nunca hay que mover ni reacomodar los elementos de abajo.

Casos de uso de una Pila

Casos de uso de una Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Deshacer** (*undo*) en un editor: cada acción se apila; deshacer es pop de la última.

Casos de uso de una Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Deshacer** (*undo*) en un editor: cada acción se apila; deshacer es pop de la última.
- **Pila de llamadas** de funciones: cada llamada se apila, y retorna en el orden **inverso** al que fue llamada — exactamente LIFO. Ya la usaste, sin saberlo, cada vez que escribiste una función recursiva.

Casos de uso de una Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Deshacer** (*undo*) en un editor: cada acción se apila; deshacer es pop de la última.
- **Pila de llamadas** de funciones: cada llamada se apila, y retorna en el orden **inverso** al que fue llamada — exactamente LIFO. Ya la usaste, sin saberlo, cada vez que escribiste una función recursiva.
- **Verificar paréntesis balanceados**: recorrer la expresión, apilando (y desapilando al ver).

Casos de uso de una Pila

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Deshacer** (*undo*) en un editor: cada acción se apila; deshacer es pop de la última.
- **Pila de llamadas** de funciones: cada llamada se apila, y retorna en el orden **inverso** al que fue llamada — exactamente LIFO. Ya la usaste, sin saberlo, cada vez que escribiste una función recursiva.
- **Verificar paréntesis balanceados**: recorrer la expresión, apilando (y desapilando al ver).
- Recorridos de grafos y árboles en profundidad (*DFS*) — lo veremos con detalle más adelante en el curso, pero la idea de "explorar y retroceder" es exactamente el comportamiento de una pila.

Ejemplo: paréntesis balanceados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Enunciado

Dado un string de (y), ¿está **balanceado**? (cada (tiene su) correspondiente, en el orden correcto).

Ejemplo: paréntesis balanceados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Enunciado

Dado un string de (y), ¿está **balanceado**? (cada (tiene su) correspondiente, en el orden correcto).

Con una pila

Recorrer el string: si es (, `push`; si es), hace falta que la pila **no** esté vacía — se hace `pop` (representa cerrar el (más reciente sin cerrar). Al final, está balanceado si la pila quedó **vacía**.

Ejemplo: paréntesis balanceados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Enunciado

Dado un string de (y), ¿está **balanceado**? (cada (tiene su) correspondiente, en el orden correcto).

Con una pila

Recorrer el string: si es (, `push`; si es), hace falta que la pila **no** esté vacía — se hace `pop` (representa cerrar el (más reciente sin cerrar). Al final, está balanceado si la pila quedó **vacía**.

¡Piénsalo!

¿Por qué falla correctamente con `")("`? ¿Y con `"(()"`?

TDA Cola (Queue)

El TDA Cola

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una **cola** (*queue*) es una colección donde se inserta por un extremo (el **final**, *back*) y se elimina por el **otro** (el **frente**, *front*).

El TDA Cola

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una **cola** (*queue*) es una colección donde se inserta por un extremo (el **final**, *back*) y se elimina por el **otro** (el **frente**, *front*).

FIFO: *First In, First Out*

El primer elemento en entrar es siempre el primero en salir — como una fila de personas: quien llegó primero, se atiende primero.

Operaciones del TDA Cola

Operación	Efecto	Complejidad exigida

Operaciones del TDA Cola

Operación	Efecto	Complejidad exigida
`enqueue(x)`	agrega x al final	$O(1)$

Operaciones del TDA Cola

Operación	Efecto	Complejidad exigida
`enqueue(x)`	agrega x al final	$O(1)$
`dequeue()`	quita el elemento del frente	$O(1)$

Operaciones del TDA Cola

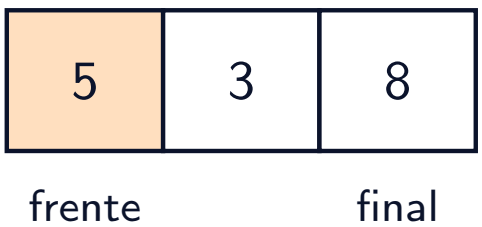
UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Efecto	Complejidad exigida
`enqueue(x)`	agrega x al final	$O(1)$
`dequeue()`	quita el elemento del frente	$O(1)$
`front()`	consulta (sin quitar) el frente	$O(1)$

Operaciones del TDA Cola

Operación	Efecto	Complejidad exigida
`enqueue(x)`	agrega x al final	$O(1)$
`dequeue()`	quita el elemento del frente	$O(1)$
`front()`	consulta (sin quitar) el frente	$O(1)$
`empty()`	¿está vacía?	$O(1)$

Visualizando enqueue y dequeue



Visualizando enqueue y dequeue



Visualizando enqueue y dequeue



Visualizando `enqueue` y `dequeue`



Insertar y eliminar ocurren en extremos **distintos** — a diferencia de la pila, donde ambos ocurren en el mismo lado.

Casos de uso de una Cola

Casos de uso de una Cola

- **Colas de atención**: impresoras, tickets de soporte, procesos esperando CPU — se atiende en el orden de llegada.

Casos de uso de una Cola

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Colas de atención**: impresoras, tickets de soporte, procesos esperando CPU — se atiende en el orden de llegada.
- ***Buffers***: datos que llegan más rápido de lo que se procesan se acumulan en una cola, y se procesan en orden.

Casos de uso de una Cola

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Colas de atención**: impresoras, tickets de soporte, procesos esperando CPU — se atiende en el orden de llegada.
- ***Buffers***: datos que llegan más rápido de lo que se procesan se acumulan en una cola, y se procesan en orden.
- Recorridos de grafos y árboles **por niveles** (*BFS*) — también lo veremos con detalle más adelante, pero ya puedes intuir por qué: se visita todo lo "más cercano" antes de avanzar, en el mismo orden en que se fue encontrando.

TDA Cola Doble (Deque)

El TDA Cola Doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una **cola doble** (*deque*, de *double-ended queue*) permite insertar **y** eliminar por **ambos** extremos.

El TDA Cola Doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una **cola doble** (*deque*, de *double-ended queue*) permite insertar **y** eliminar por **ambos** extremos.

¡Piénsalo!

Si una pila usa un extremo, y una cola usa los dos extremos pero de forma "fija" (uno para entrar, otro para salir), ¿qué relación crees que tiene el deque con ambas?

Operaciones del TDA Cola Doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Efecto	Complejidad exigida

Operaciones del TDA Cola Doble

Operación	Efecto	Complejidad exigida
`push_front(x)`	agrega x al inicio	$O(1)$

Operaciones del TDA Cola Doble

Operación	Efecto	Complejidad exigida
`push_front(x)`	agrega x al inicio	$O(1)$
`push_back(x)`	agrega x al final	$O(1)$

Operaciones del TDA Cola Doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Efecto	Complejidad exigida
`push_front(x)`	agrega x al inicio	$O(1)$
`push_back(x)`	agrega x al final	$O(1)$
`pop_front()`	quita del inicio	$O(1)$

Operaciones del TDA Cola Doble

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Efecto	Complejidad exigida
`push_front(x)`	agrega x al inicio	$O(1)$
`push_back(x)`	agrega x al final	$O(1)$
`pop_front()`	quita del inicio	$O(1)$
`pop_back()`	quita del final	$O(1)$

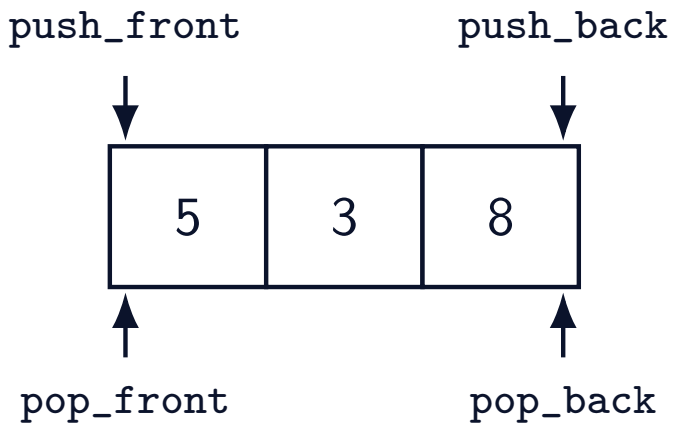
Operaciones del TDA Cola Doble

Operación	Efecto	Complejidad exigida
`push_front(x)`	agrega x al inicio	$O(1)$
`push_back(x)`	agrega x al final	$O(1)$
`pop_front()`	quita del inicio	$O(1)$
`pop_back()`	quita del final	$O(1)$

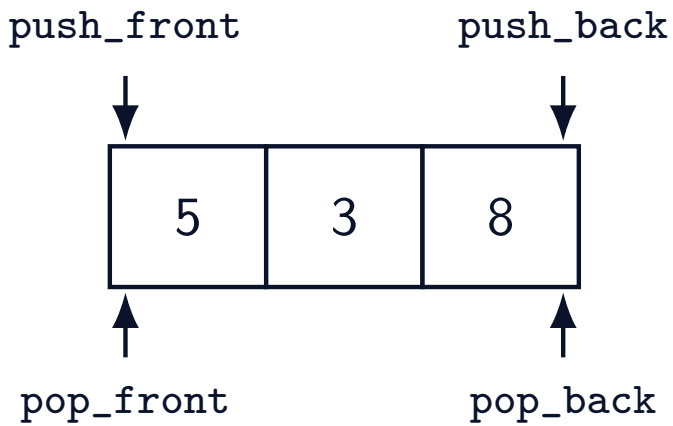
Deque generaliza a los otros dos

Usando solo `push_back`/`pop_back`: es una **pila**. Usando `push_back`/`pop_front`: es una **cola**. El deque no elimina la necesidad de los otros dos TDAs — **restringir** la interfaz sigue siendo útil, como veremos al final de la clase.

Visualizando el deque: ambos extremos libres



Visualizando el deque: ambos extremos libres



Las cuatro operaciones tocan solo un extremo a la vez: nunca hace falta desplazar los elementos del medio.

Casos de uso de un Deque

Casos de uso de un Deque

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Historial de navegación**: "atrás" y "adelante" funcionan sobre los dos extremos de una secuencia de páginas visitadas.

Casos de uso de un Deque

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Historial de navegación**: "atrás" y "adelante" funcionan sobre los dos extremos de una secuencia de páginas visitadas.
- **Verificar palíndromos**: comparar el elemento del frente con el del final, y achicar el deque desde ambos lados a la vez.

Casos de uso de un Deque

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Historial de navegación**: "atrás" y "adelante" funcionan sobre los dos extremos de una secuencia de páginas visitadas.
- **Verificar palíndromos**: comparar el elemento del frente con el del final, y achicar el deque desde ambos lados a la vez.
- **Ventanas deslizantes** (*sliding window*) sobre un arreglo, para mantener — por ejemplo — el máximo del rango actual: técnica que verás en detalle en cursos posteriores, pero que depende exactamente de poder insertar y eliminar por ambos extremos en $O(1)$.

Comparando los tres

Tabla comparativa



	Pila	Cola	Deque

Tabla comparativa



	Pila	Cola	Deque
Extremos de inserción	1	1 (fijo)	2

Tabla comparativa

	Pila	Cola	Deque
Extremos de inserción	1	1 (fijo)	2
Extremos de eliminación	1 (el mismo)	1 (el otro)	2

Tabla comparativa

	Pila	**Cola**	**Deque**
Extremos de inserción	1	1 (fijo)	2
Extremos de eliminación	1 (el mismo)	1 (el otro)	2
Orden de salida	LIFO	FIFO	cualquiera de los dos

Tabla comparativa

	Pila	Cola	Deque
Extremos de inserción	1	1 (fijo)	2
Extremos de eliminación	1 (el mismo)	1 (el otro)	2
Orden de salida	LIFO	FIFO	cualquiera de los dos

¡Piénsalo!

Si el deque puede hacer todo lo que hacen pila y cola, ¿por qué seguir usando el TDA más restringido en vez de usar siempre un deque?

La respuesta (adelanto de la próxima clase)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una interfaz más restringida comunica intención

Usar una **pila** en tu código deja claro, para quien lo lea después, que el orden de acceso es LIFO — sin tener que confiar en que "nadie use `push_front` por error".

La respuesta (adelanto de la próxima clase)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una interfaz más restringida comunica intención

Usar una **pila** en tu código deja claro, para quien lo lea después, que el orden de acceso es LIFO — sin tener que confiar en que "nadie use `push_front` por error".

Y también depende de la implementación

No todas las formas de implementar un deque logran que **las cuatro** operaciones sean $O(1)$ con la misma facilidad que una pila o una cola — la próxima clase implementamos los tres, y vemos exactamente dónde está la dificultad.

Resumen de la sesión



Resumen de la sesión

- Una **pila** (LIFO) inserta y elimina por un mismo extremo, el tope; una **cola** (FIFO) inserta por un extremo y elimina por el otro.

Resumen de la sesión

- Una **pila** (LIFO) inserta y elimina por un mismo extremo, el tope; una **cola** (FIFO) inserta por un extremo y elimina por el otro.
- Un **deque** generaliza a ambos, permitiendo insertar y eliminar por los dos extremos.

Resumen de la sesión

- Una **pila** (LIFO) inserta y elimina por un mismo extremo, el tope; una **cola** (FIFO) inserta por un extremo y elimina por el otro.
- Un **deque** generaliza a ambos, permitiendo insertar y eliminar por los dos extremos.
- El contrato de los tres exige $O(1)$ en todas sus operaciones — todavía no dijimos **cómo** lograrlo.

Resumen de la sesión

- Una **pila** (LIFO) inserta y elimina por un mismo extremo, el tope; una **cola** (FIFO) inserta por un extremo y elimina por el otro.
- Un **deque** generaliza a ambos, permitiendo insertar y eliminar por los dos extremos.
- El contrato de los tres exige $O(1)$ en todas sus operaciones — todavía no dijimos **cómo** lograrlo.
- Aparecen por todas partes: deshacer, paréntesis balanceados, colas de atención, historial de navegación, y (más adelante) recorridos de grafos.

Resumen de la sesión

- Una **pila** (LIFO) inserta y elimina por un mismo extremo, el tope; una **cola** (FIFO) inserta por un extremo y elimina por el otro.
- Un **deque** generaliza a ambos, permitiendo insertar y eliminar por los dos extremos.
- El contrato de los tres exige $O(1)$ en todas sus operaciones — todavía no dijimos **cómo** lograrlo.
- Aparecen por todas partes: deshacer, paréntesis balanceados, colas de atención, historial de navegación, y (más adelante) recorridos de grafos.
- **Próxima clase:** implementamos los tres TDAs. Ya conocemos arreglos, `vector` y listas enlazadas — veremos con cuál conviene implementar cada uno, y por qué.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias